Module 8 — AI-Assisted Release Readiness: Hands-On Lab

Oct 30, 2026 · @Durai

Overview

This builds the Module 8 case study end to end: score a release's risk, auto-draft its release notes, and surface a go/no-go scorecard — as five small PowerShell scripts you run from the terminal, the same pattern as Module 7's `summarize-diff.ps1` (not wired into the pipeline; a live, narrated run is the point).

It reuses your existing sandbox rather than standing up anything new:

- **Azure DevOps org** `traininguser158` — assumed project: **pharmacy-api** (the Module 4/5 app), since it already has the four signal families this module needs: SonarCloud quality gates and JaCoCo coverage (Module 4), and a staging approval gate with deployment history (Module 5). If pharmacy-api turns out to live under a different org, or you'd rather use PharmacyApi-DevSecOps-Demo instead, the scripts below only need the `$Org` / `$Project` / `$Repo` variables changed.
- **Azure OpenAI** `aoai-pharmacyapi-demo`, deployment `gpt-5-4-mini` — the same resource and deployment Module 7's `summarize-diff.ps1` already calls.

New for this lab: one Azure DevOps Personal Access Token (PAT) with read access to Code, Build and Work Items, so the scripts can pull PRs, builds and work items via the REST API.

Prerequisites

Already in place	New for this lab
Azure DevOps org `traininguser158`, pharmacy-api project with PR/build history	A PAT scoped to Code (Read), Build (Read) and Work Items (Read) — Step 1
Azure OpenAI resource `aoai-pharmacyapi-demo`, `gpt-5-4-mini` deployment	A local `release-readiness/` folder to hold the five scripts and their output files

Already in place	New for this lab
PowerShell + the Azure OpenAI call pattern from Module 7's `summarize-diff.ps1`	`$env:AOAI_ENDPOINT` / `$env:AOAI_KEY` set in your shell (reuse the same values Module 7 used) if you haven't kept them from that session
SonarCloud quality-gate + JaCoCo coverage results on pharmacy-api (Module 4)	—
A staging approval gate with deployment history (Module 5)	—

If `$env:AOAI_ENDPOINT` and `$env:AOAI_KEY` aren't still set from Module 7, pull them again: Azure Portal → `aoai-pharmacyapi-demo` → **Keys and Endpoint** (left nav, under Resource Management).

Step 1 — Create the Azure DevOps PAT and verify REST API access

1. In Azure DevOps, click your profile icon (top right) → **Personal access tokens**.
2. **+ New Token**.
3. **Name**: `module8-release-readiness`. **Organization**: `traininguser158`. **Expiration**: 30 days is enough for the sandbox.
4. **Scopes** → **Custom defined** → tick: **Code → Read**, **Build → Read**, **Work Items → Read**.
5. **Create**, then copy the token immediately — Azure DevOps shows it once.
6. In PowerShell, set it as an environment variable for this session (don't hardcode it into any script):

```
$env:ADO_PAT = "<paste the token>"
$Org     = "traininguser158"
$Project = "pharmacy-api"
$Repo    = "pharmacy-api"   # update if the repo name differs from the
project name
```

7. Sanity-check the PAT works before writing any scripts — list the repo's completed pull requests:

```
$authHeader = @{ Authorization = "Basic " +
[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes(":$env:ADO_PAT")) }
$url =
"https://dev.azure.com/$Org/$Project/_apis/git/repositories/$Repo/pullrequest
s?searchCriteria.status=completed&`$top=5&api-version=7.1"
Invoke-RestMethod -Uri $url -Headers $authHeader -Method Get | Select-Object
-ExpandProperty value | Select-Object pullRequestId, title, closedDate
```

If that returns a short table of recent PRs, the PAT and REST access are both good — move on to Step 2. A 401/403 means a scope or org mismatch; re-check Step 4.

Step 2 — Script 1: AI change-risk score

Pulls every PR merged since a given date, summarises what changed, and asks Azure OpenAI for a risk band with a rationale. Save as `release-readiness/risk-score.ps1`:

```
param(
    [string]$Org = "traininguser158",
    [string]$Project = "pharmacy-api",
    [string]$Repo = "pharmacy-api",
    [datetime]$Since = (Get-Date).AddDays(-14)
)

$authHeader = @{ Authorization = "Basic " +
[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes(":$env:ADO_PAT")) }
$base = "https://dev.azure.com/$Org/$Project/_apis"

# 1. Pull completed PRs since $Since
$prs = Invoke-RestMethod -Uri "$base/git/repositories/$Repo/pullrequests?
searchCriteria.status=completed&`$top=50&api-version=7.1" -Headers
$authHeader |
    Select-Object -ExpandProperty value |
    Where-Object { $_.closedDate -and [datetime]$_.closedDate -ge $Since }

if (-not $prs) { Write-Host "No PRs merged since $Since"; exit }

# 2. For each PR, pull its changed files (iteration 1 is usually enough for a
size/shape signal)
$changeSummary = foreach ($pr in $prs) {
    $iterations = Invoke-RestMethod -Uri
"$base/git/repositories/$Repo/pullrequests/$($pr.pullRequestId)/iterations?
api-version=7.1" -Headers $authHeader
```

```
    $lastIter = $iterations.value | Select-Object -Last 1
    $changes = Invoke-RestMethod -Uri
"$base/git/repositories/$Repo/pullrequests/$($pr.pullRequestId)/iterations/$(
$lastIter.id)/changes?api-version=7.1" -Headers $authHeader
    [pscustomobject]@{
        PR = $pr.pullRequestId
        Title = $pr.title
        FilesChanged = $changes.changeEntries.Count
        Paths = ($changes.changeEntries.item.path -join ", ")
    }
}

$changeSummary | Format-Table -AutoSize

# 3. Known hotspots — hardcoded for the lab; in a real setup this comes from
an incident log
$hotspots = @("OrderService.java", "PaymentController.java")

$prompt = @"
You are a release-risk assistant. Given this list of PRs merged in the last
release window, score the overall change-set risk as LOW, MEDIUM or HIGH, and
explain your reasoning in 3-4 sentences. Flag any PR that touches a known
hotspot file.

Known hotspot files: $($hotspots -join ', ')

PRs:
$($changeSummary | ConvertTo-Json -Depth 3)
"@

$body = @{
    messages = @(
        @{ role = "system"; content = "You are a concise, precise release-
risk assistant for a DevOps pipeline." },
        @{ role = "user"; content = $prompt }
    )
    max_completion_tokens = 500
} | ConvertTo-Json -Depth 5

$aoaiUrl = "$($env:AOAI_ENDPOINT)/openai/deployments/gpt-5-4-
mini/chat/completions?api-version=2024-10-21"
$result = Invoke-RestMethod -Uri $aoaiUrl -Headers @{ "api-key" =
$env:AOAI_KEY } -Method Post -ContentType "application/json" -Body $body
```

```
$riskReport = $result.choices[0].message.content
Write-Host "`n--- RISK REPORT ---`n$riskReport"
$riskReport | Out-File "risk-score.md"
```

Run it with `./risk-score.ps1`. It prints the PR table, then the AI's risk band and rationale, and saves the rationale to `risk-score.md` for Script 5 to pick up later.

Step 3 — Script 2: AI-drafted release notes

Same PR pull as Script 1, but asks for a structured changelog instead of a risk score. Save as `release-readiness/release-notes.ps1`:

```
param(
    [string]$Org = "traininguser158",
    [string]$Project = "pharmacy-api",
    [string]$Repo = "pharmacy-api",
    [datetime]$Since = (Get-Date).AddDays(-14),
    [string]$Version = "v2.4.0"
)

$authHeader = @{ Authorization = "Basic " +
[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes(":$env:ADO_PAT")) }
$base = "https://dev.azure.com/$Org/$Project/_apis"

$prs = Invoke-RestMethod -Uri "$base/git/repositories/$Repo/pullrequests?
searchCriteria.status=completed&`$top=50&api-version=7.1" -Headers
$authHeader |
    Select-Object -ExpandProperty value |
    Where-Object { $_.closedDate -and [datetime]$_.closedDate -ge $Since } |
    Select-Object pullRequestId, title, description, closedDate

if (-not $prs) { Write-Host "No PRs merged since $Since"; exit }

$prompt = @"
Draft a release changelog for $Version from these merged PR titles and
descriptions. Group into Features, Fixes and Breaking changes (use 'None
detected' if a group is empty — and note it should still be verified by a
human). One line per PR, plain and specific, no marketing language. Output as
markdown.

PRs:
```

```
$($prs | ConvertTo-Json -Depth 3)
"@

$body = @{
    messages = @(
        @{ role = "system"; content = "You write concise, accurate release
notes for engineers, not end users." },
        @{ role = "user"; content = $prompt }
    )
    max_completion_tokens = 600
} | ConvertTo-Json -Depth 5

$aoaiUrl = "$($env:AOAI_ENDPOINT)/openai/deployments/gpt-5-4-
mini/chat/completions?api-version=2024-10-21"
$result = Invoke-RestMethod -Uri $aoaiUrl -Headers @{ "api-key" =
$env:AOAI_KEY } -Method Post -ContentType "application/json" -Body $body

$notes = $result.choices[0].message.content
Write-Host "`n--- DRAFT RELEASE NOTES ---`n$notes"
$notes | Out-File "release-notes-draft.md"
```

Run with `./release-notes.ps1`. Open `release-notes-draft.md` afterwards and read it as the human editor would — this is the moment in the live demo to deliberately show one thing you'd correct or reword before it ships, so the "you edit, not author" point lands.

Step 4 — Script 3: AI failure-triage hint

Takes one failed build, pulls its log, and asks for a likely root cause. You'll need a real failed build id to demo this — easiest is to deliberately break a test in a throwaway branch and push it, or reuse a build that already failed during earlier module practice. Save as `release-readiness/triage.ps1`:

```
param(
    [string]$Org = "traininguser158",
    [string]$Project = "pharmacy-api",
    [Parameter(Mandatory=$true)][int]$BuildId
)

$authHeader = @{ Authorization = "Basic " +
[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes(":$env:ADO_PAT")) }
$base = "https://dev.azure.com/$Org/$Project/_apis"
```

```
# 1. Confirm the build actually failed
$build = Invoke-RestMethod -Uri "$base/build/builds/$($BuildId)?api-
version=7.1" -Headers $authHeader
Write-Host "Build $BuildId status: $($build.result)"

# 2. Pull the timeline to find the failed task's log id
$timeline = Invoke-RestMethod -Uri "$base/build/builds/$BuildId/timeline?api-
version=7.1" -Headers $authHeader
$failedStep = $timeline.records | Where-Object { $_.result -eq "failed" -and
$_.log } | Select-Object -First 1

if (-not $failedStep) { Write-Host "No failed step with a log found on build
$BuildId"; exit }

# 3. Pull that step's log text (tail it — logs can be long)
$logText = Invoke-RestMethod -Uri "$($failedStep.log.url)" -Headers
$authHeader
$logTail = ($logText -split "`n" | Select-Object -Last 120) -join "`n"

$prompt = @"
A CI build step failed. Here is the tail of its log. In 3-5 sentences: (1)
state the most likely root cause in plain language, (2) point to the specific
line or error that supports it, (3) suggest one concrete next step to confirm
it. Do not suggest a code fix — only a diagnosis and a way to confirm it.

Failed step: $($failedStep.name)

Log tail:
$logTail
"@

$body = @{
    messages = @(
        @{ role = "system"; content = "You are a build-failure triage
assistant. You diagnose, you never auto-fix." },
        @{ role = "user"; content = $prompt }
    )
    max_completion_tokens = 400
} | ConvertTo-Json -Depth 5

$aoaiUrl = "$($env:AOAI_ENDPOINT)/openai/deployments/gpt-5-4-
mini/chat/completions?api-version=2024-10-21"
$result = Invoke-RestMethod -Uri $aoaiUrl -Headers @{ "api-key" =
```

```
$env:AOAI_KEY } -Method Post -ContentType "application/json" -Body $body

$hint = $result.choices[0].message.content
Write-Host "`n--- TRIAGE HINT (build $BuildId) ---`n$hint"
$hint | Out-File "triage-hint.md"
```

Run with `./triage.ps1 -BuildId <id>`. In the live session, this is the slide-9 "human confirms" moment made concrete — read the hint aloud, then say out loud whether you agree with it before accepting it, so the triage slide's guardrail isn't just a line on a slide.

Step 5 — Script 4: simple anomaly check

A deliberately lightweight version — full DORA tracking needs more plumbing than the lab window allows, so this pulls what the Build API gives for free: recent build durations and pass/fail outcomes, compares against a rolling baseline, and flags drift. Save as `release-readiness/anomaly-check.ps1`:

```
param(
    [string]$Org = "traininguser158",
    [string]$Project = "pharmacy-api",
    [int]$LookbackCount = 20
)

$authHeader = @{ Authorization = "Basic " +
[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes(":$env:ADO_PAT")) }
$base = "https://dev.azure.com/$Org/$Project/_apis"

$builds = Invoke-RestMethod -Uri "$base/build/builds?
`$top=$LookbackCount&queryOrder=finishTimeDescending&api-version=7.1" -
Headers $authHeader |
    Select-Object -ExpandProperty value |
    Where-Object { $_.finishTime } |
    Select-Object buildNumber, result, startTime, finishTime,
        @{ n = "durationMin"; e = { ([datetime]$_.finishTime -
[datetime]$_.startTime).TotalMinutes } }

if ($builds.Count -lt 5) { Write-Host "Not enough build history for a
baseline — need at least 5 runs"; exit }

$durations = $builds.durationMin
$baseline = ($durations | Measure-Object -Average).Average
$latest = $durations[0]
```

```
$deviationPct = [math]::Round((($latest - $baseline) / $baseline) * 100, 1)

$failRate = [math]::Round((($builds | Where-Object { $_.result -eq "failed"
}).Count / $builds.Count) * 100, 1)

Write-Host "Baseline avg duration: $([math]::Round($baseline,1)) min over
$($builds.Count) builds"
Write-Host "Latest build duration: $([math]::Round($latest,1)) min
($deviationPct% vs. baseline)"
Write-Host "Recent fail rate: $failRate%"

$flags = @()
if ([math]::Abs($deviationPct) -gt 30) { $flags += "Build duration drifted
$deviationPct% from baseline" }
if ($failRate -gt 20) { $flags += "Fail rate over the last $($builds.Count)
builds is $failRate%" }

$report = [pscustomobject]@{
    BaselineDurationMin = [math]::Round($baseline,1)
    LatestDurationMin = [math]::Round($latest,1)
    DeviationPct = $deviationPct
    RecentFailRatePct = $failRate
    Flags = if ($flags) { $flags } else { @("None") }
}
$report | ConvertTo-Json | Out-File "anomaly-report.json"
$report
```

Run with `./anomaly-check.ps1`. The 30%/20% thresholds are placeholders — call that out in the session: real thresholds come from watching a team's own baseline for a few weeks, not a number picked on day one.

Step 6 — Script 5: the go/no-go scorecard

Reads the outputs of Scripts 1, 2 and 4, adds a manual quality-gate check (pulled from SonarCloud, or just typed in — see note below), and asks Azure OpenAI to write the final scorecard narrative. Save as `release-readiness/scorecard.ps1`:

```
param(
    [ValidateSet("pass","fail")][string]$QualityGate = "pass",
    [string]$CoveragePct = "62%"
)
```

```
$riskReport = if (Test-Path "risk-score.md") { Get-Content "risk-score.md" -
Raw } else { "Not run — see risk-score.ps1" }
$releaseNotes = if (Test-Path "release-notes-draft.md") { Get-Content
"release-notes-draft.md" -Raw } else { "Not run — see release-notes.ps1" }
$anomalyReport = if (Test-Path "anomaly-report.json") { Get-Content "anomaly-
report.json" -Raw } else { "Not run — see anomaly-check.ps1" }

$prompt = @"
You are producing a release-readiness scorecard for a human approver. Combine
these four signals into ONE markdown scorecard with this exact structure:

1. A 'Readiness Score' out of 100 (your own judgement from the inputs)
2. A recommendation: GO, GO WITH CONDITIONS, or NO-GO — never a plain GO if
the quality gate failed
3. A short table: Coverage & Gates | Change Risk | Traceability | Pipeline
Health — one line each
4. A 2-3 sentence rationale
5. A closing line: 'This is a recommendation. A human reviewer makes the
final call.'

Inputs:

Quality gate: $QualityGate, coverage: $CoveragePct

Change-risk report:
$riskReport

Draft release notes:
$releaseNotes

Anomaly report:
$anomalyReport
"@

$body = @{
    messages = @(
        @{ role = "system"; content = "You produce release-readiness
scorecards. You never recommend GO when the quality gate has failed, and you
always end with the human-sign-off line verbatim." },
        @{ role = "user"; content = $prompt }
    )
    max_completion_tokens = 700
} | ConvertTo-Json -Depth 5
```

```
$aoaiUrl = "$($env:AOAI_ENDPOINT)/openai/deployments/gpt-5-4-
mini/chat/completions?api-version=2024-10-21"
$result = Invoke-RestMethod -Uri $aoaiUrl -Headers @{ "api-key" =
$env:AOAI_KEY } -Method Post -ContentType "application/json" -Body $body

$scorecard = $result.choices[0].message.content
Write-Host "`n--- RELEASE-READINESS SCORECARD ---`n$scorecard"
$scorecard | Out-File "scorecard.md"
```

Run with `./scorecard.ps1` (or `-QualityGate fail` to demo the NO-GO path on purpose — a good live-session beat, since a clean GO every time undersells the point).

On `$QualityGate` / `$CoveragePct` being typed in rather than pulled live: wiring this to SonarCloud's API (`GET https://sonarcloud.io/api/qualitygates/project_status?projectKey=<key>`, auth with a SonarCloud token) is a natural next step but adds a second credential and a second API shape to the live demo. For the first run-through, typing the two values in keeps the session focused on the AI-combination step, which is the actual learning objective here; wire the live SonarCloud call in afterwards if time allows.

Step 7 — End-to-end dry run

With the PAT and `$env:AOAI_*` variables set, run the four scripts in sequence against pharmacy-api's real history:

```
cd release-readiness
./risk-score.ps1
./release-notes.ps1
./anomaly-check.ps1
./scorecard.ps1
```

Check, in order:

1. `risk-score.ps1` prints a PR table, then a risk band (LOW/MEDIUM/HIGH) with a rationale that actually references the PRs pulled — not a generic answer.
2. `release-notes.ps1` produces a changelog grouped into Features/Fixes/Breaking changes that matches what you'd expect from the PR titles.
3. `anomaly-check.ps1` prints a baseline duration, a deviation percentage and a fail rate — needs at least 5 prior builds in the project's history to run at all.

4. `scorecard.ps1` produces one markdown file combining all three, ending in the human-sign-off line.

If `pharmacy-api` doesn't have 14 days of recent PR activity, widen `-Since` on Scripts 1 and 2 (e.g. `-Since (Get-Date).AddDays(-60)`) rather than inventing data — real output, even from an older window, teaches the point better than a script that silently falls back to placeholders.

For the live session, run this in front of the class exactly as above — four real API round-trips to real Azure DevOps data, not pre-baked output. That's the demonstration: the scorecard is assembled from what the pipeline already knows, not from a canned example.

Open decisions

- [ ] Confirm pharmacy-api's actual org and project names — this doc assumes `traininguser158` / `pharmacy-api` / repo `pharmacy-api`
- [ ] Confirm `$env:AOAI_ENDPOINT` / `$env:AOAI_KEY` are still valid, or re-pull them from the Azure Portal
- [ ] Decide whether to wire Script 5's quality-gate check to the live SonarCloud API or keep it as typed-in parameters for the first run-through
- [ ] Pick (or deliberately break) a failed build to demo Script 3 (triage) against — it needs a real failure, not a clean build
- [ ] Decide whether the four scripts run live from a laptop terminal (matching Module 7's approach) or get wrapped into a sixth "run everything" script for the session
- [ ] Decide whether Demo 8 and the Capstone share the same `release-readiness/` scripts, or the Capstone gets its own copy wired into the pipeline itself